



**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

**UTC Summer School: Advanced Sequencing
informatics and Genome assembly**

Genome Assembly Assignment

Made By:

Chia Rui Hsien, Ethan

Table of Contents

1 Aims	3
2 Objectives.....	3
3 Quality Control on Raw Reads	3
3.1 FastQC Analysis.....	3
3.2 K-mer Analysis	5
4 De novo Assembly	7
4.1 MaSuRCA.....	7
4.2 HybridSPAdes	10
4.3 DBG2OLC with SPAdes	11
4.4 Pilon	14
5 Assembly Quality Control.....	15
5.1 gnx-tools and quast.....	15
5.2 Merqury	17
5.3 Conclusion	19
References.....	20

1 Aims

The aim of this assignment is to perform a de-novo genome assembly and report your workflow and findings.

As the bioinformatics leader in your research group, you have been asked to perform a de novo genome assembly for an “unknown” organism. These are Illumina PE 101x2 with insert size 350bps +/-50 as well as some PacBio long reads.

2 Objectives

1. Perform a QC based on the raw reads (only for the short reads), and perform any quality filtering/trimming if necessary - Estimate the genome size by performing a k-mer analysis and report the contig N50 [5 marks]
2. Perform a de-novo assembly of your reads using best practices as covered throughout this module. You can use assemblers covered during this week’s practicals, or other assemblers which are compatible with the sequencing data you have been provided with – report all your attempts (at least three attempts are required, but feel free to do more! – Make sure you include the code for your attempts as a qsub script(s)) and justify your choice of parameters. Choose the best assembly for step 4 below [70 marks]
3. Perform the assembly quality control according to the best practices covered during this module. Your assessment should include statistical metrics (N50, longest contig/scaffold, etc.) as well as comparative quality evaluation between your best 3 attempts – Justify your decision in picking the best assembly [25 Marks]

3 Quality Control on Raw Reads

Quality control was performed on the Illumina PE 101x2 with insert size 350bps +/-50 reads.

```
# Setting up directories and downloading sample_data
cd
mkdir assignment
cd assignment
cp -R /volpilcam/ressources/0LOGICIELS/SHARED/IBIX/IBIX-
SYS_Data_for_Assignment
mv IBIX-SYS_Data_for_Assignment sample_data
```

3.1 FastQC Analysis

```
fastqc #Runs FastQC
```

Open the HS7_R1.fastq.gz and HS7_R2.fastq.gz files in FastQC

Measure	Value	Value
Filename	HS7_R1.fastq.gz	HS7_R2.fastq.gz
File type	Conventional base calls	Conventional base calls
Encoding	Sanger / Illumina 1.9	Sanger / Illumina 1.9
Total Sequences	2475000	2475000
Total Bases	249.9 Mbp	249.9 Mbp
Sequences flagged as poor quality	0	0
Sequence length	101	101
%GC	36	36

Figure 1: Image of Basic Sequence Statistics from FASTQC for HS7_R1.fastq.gz and HS7_R2.fastq.gz respectively

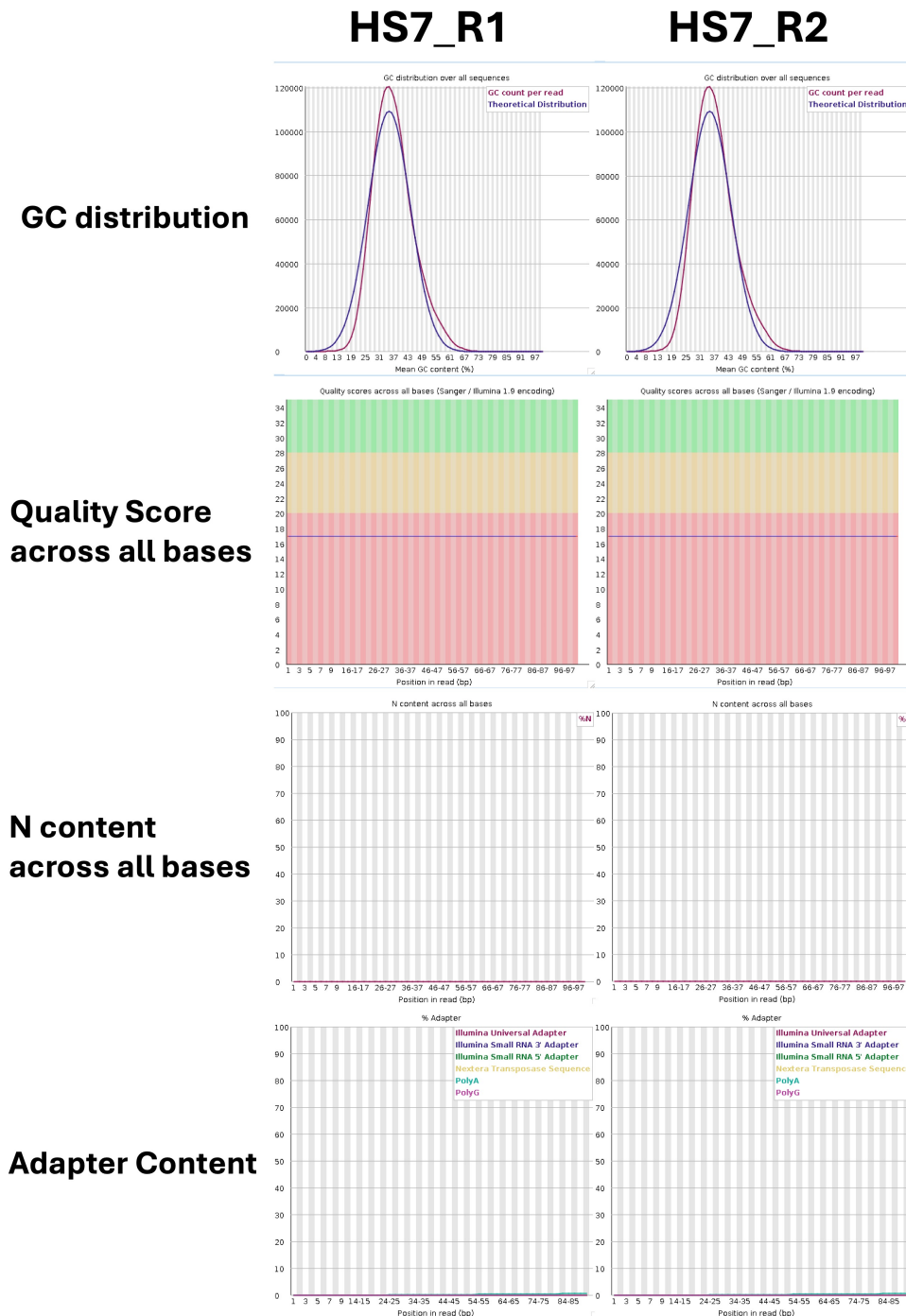


Figure 2: Images from FASTQC for HS7_R1.fastq.gz and HS7_R2.fastq.gz representing GC distribution, quality score across bases, N content across all bases and adapter content respectively

From Figure 1 and 2, we can observe that the short reads are of good quality because there are no sequences that have been flagged as poor quality and the quality score across all bases is consistent at 17. The short reads are 101bp in length which is normal for short reads. There is also little to no adapter content present. Therefore, no filtering or trimming of the short reads is necessary.

Both figures also support the fact that both files are paired end reads since there are an equal number of sequences of 2475000 for both. The files are also likely artificially generated since the quality scores for both files are constant at 17 which is not normal for sequenced reads.

3.2 K-mer Analysis

K-mer analysis was performed using Jellyfish (Marçais & Kingsford, 2011).

```
# Setting up directories and bash scripts for k-mer analysis
cd
cd assignments
mkdir QC_analysis
cd QC_analysis
touch run_jelly.sh
chmod 755 run_jelly.sh
vim run_jelly.sh
```

Bash script: run_jelly.sh

```
#!/bin/bash
# Defining Variables
R1="/voletu/users/echiarui/assignment/sample_data/HS7_R1.fastq.gz"
R2="/voletu/users/echiarui/assignment/sample_data/HS7_R2.fastq.gz"
out_dir="/voletu/users/echiarui/assignment/QC_analysis"

# Code to run Jellyfish to find genome size from the kmer spectrum
produced
zcat "$R1" "$R2" | conda run -n bioinformatics jellyfish count \
-t 2 \
-C \
-m 19 \
-s 1G \
-o "${out_dir}/R12corr_19mer_out"

conda run -n bioinformatics jellyfish histo \
-o "${out_dir}/R12_19mer.histo" \
"${out_dir}/R12corr_19mer_out"
```

```
# Command to run in the terminal
nohup bash run_jelly.sh > jelly.log 2>&1 &
```

Run the following code in the terminal to view the graphs

```
# Code to generate k-mer graph
R
df19 <- read.table("R12_19mer.histo")
plot(df19[1:200,], type="l")
plot(df19[40:200,], type="l")
totKmers =
sum(as.numeric(df19[40:nrow(df19),1]*df19[40:nrow(df19),2]))
totKmers
G = totKmers/56
G
sCPeak <- sum(as.numeric(df19[40:80,1]*df19[40:80,2]))/56
poisTh <- dpois(40:200,56)*sCPeak
plot(poisTh, type="l", lty=2, col="blue")
lines(df19[40:200,], type="l")
```

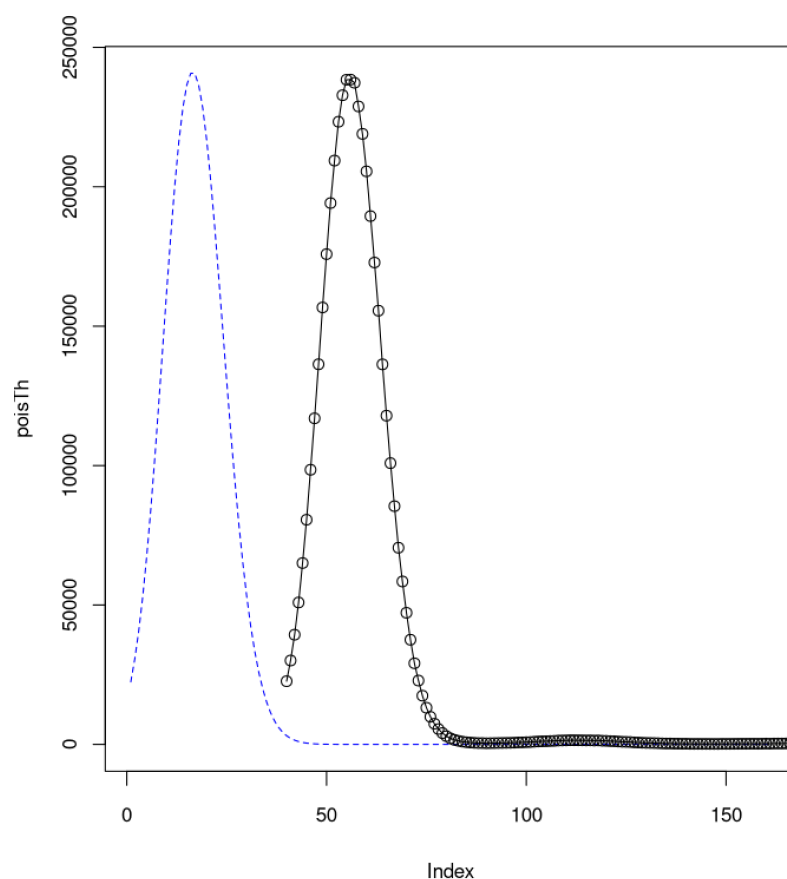


Figure 3: Graph of K-mers for short reads generated using Jellyfish and R. The blue graph represents the theoretical distribution and the black graph represents the actual distribution

From Figure 3, the maximum point of the graph was found to be the 56th index. Using this information, the total k-mer count was calculated to be 276,321,644 and the estimated genome size is 4,934,315 (4.93 Mb).

```
# Code to run gn timer
conda deactivate
java -jar /opt/bioinformatics/gnx-tools/gnx.jar
~/assignment/sample_data/HS7_R1.fastq.gz

java -jar /opt/bioinformatics/gnx-tools/gnx.jar
~/assignment/sample_data/HS7_R2.fastq.gz
```

```
(base) flux:echiarui:1003:~/assignment/sample_data> java -jar /opt/bioinformatics/gnx-tools/gnx.jar HS7_R1.fastq.gz
Results for HS7_R1.fastq.gz
Total number of sequences:      1
Total length of sequences:      253948358 bp
Shortest sequence length :      253948358 bp
Longest sequence length :      253948358 bp
Total number of Ns in sequences: 0
NS0: 253948358 (1 sequences) (253948358 bp combined)

(base) flux:echiarui:1004:~/assignment/sample_data> java -jar /opt/bioinformatics/gnx-tools/gnx.jar HS7_R2.fastq.gz
Results for HS7_R2.fastq.gz
Total number of sequences:      1
Total length of sequences:      253948358 bp
Shortest sequence length :      253948358 bp
Longest sequence length :      253948358 bp
Total number of Ns in sequences: 0
NS0: 253948358 (1 sequences) (253948358 bp combined)
```

Figure 4: Screenshot of the gn timer output for HS7_R1.fastq.gz and HS7_R2.fastq.gz files

Using gn timer, the contig N50 for R1 and R2 was found to be 253,948,358 which is close to the total k-mer count of 276,321,644.

4 De novo Assembly

De novo assembly was performed using three different short-long read hybrid assembly tools, namely MaSuRCA, HybridSPAdes and DBG2OLC because the data provided was in the form of highly accurate, paired end short reads from Illumina and error prone long read data from PacBio which these three hybrid assemblers were built to handle.

Short-long read hybrid assembly tools aim to utilise the strengths of both short, highly accurate reads and long, error prone reads to increase the accuracy of the de novo genome assembly generated from both. The goal of this is to identify which of the three hybrid assembly methods is the best for this genome.

The resulting assemblies were subjected to assembly polishing using pilon.

4.1 MaSuRCA

MaSuRCA (Maryland Super-Read Celera Assembler) is a hybrid genome assembler designed to effectively assemble large and repetitive genomes.

MaSuRCA addresses the key challenge of long-read sequencing technologies (like PacBio) having a high error rate. This high error rate makes it difficult to use long-reads solely for complex genomes. MaSuRCA handles this issue by using a hybrid genome assembly method which combines short, highly accurate reads with long, error prone reads. The algorithm

transforms noisy long reads into a set of mega-reads, which are both long and highly accurate. These mega-reads are then used for the final assembly (Zimin et al., 2017).

MaSuRCA leverages short-read data to correct the long-read data, creating a high-quality input that allows the assembler to span repetitive regions and produce a much more complete and accurate genome.

Below illustrates the code that was used to run MaSuRCA on the short and long reads.

```
# Defining the conda environment and creating the directory
cd
conda activate bioinformatics
mkdir -p ~/assignment/test1
mkdir -p ~/assignment/test1/masurca
cd ~/assignment/test1/masurca

# Code to generate masurca configuration file
masurca -g masurca_config

# Code to edit masurca_config
vim masurca_config
```

Edits made to masurca_config:

```
PE= pe 350 50
/voletu/users/echiarui/assignment/sample_data/HS7_R1.fastq.gz
/voletu/users/echiarui/assignment/sample_data/HS7_R2.fastq.gz
```

The line starting with PACBIO, remove the #, and replace it with:

```
PACBIO=/voletu/users/echiarui/assignment/sample_data/HS7_pacbioData.fastq.gz
```

Set the number of threads to use:

```
NUM_THREADS=4
```

Commented out JUMP and OTHERS with a #

Reason for Parameters: PE was changed to 350 and 50 to account for the insert size of the reads (350bp) and the deviation (± 50). This is also to indicate that HS7_R1 and HS7_R2 are paired end reads. PACBIO was changed to the HS7_pacbioData provided.

```
# Run command in terminal
export PATH=$PATH:./
```

```
# Loading and editing masurca scripts
cp
/volpilcam/ressources/0LOGICIELS/SHARED/IBIX/assemble_patched.sh
./
cp
/volpilcam/ressources/0LOGICIELS/SHARED/IBIX/mega_reads_assemble_c
luster2_patched.sh ./

# Command to edit assemble_patched.sh
vim assemble_patched.sh
```

Edits made to assemble_patched.sh:

- Changed username from fmohareb to echiarui
- CONFIG_PATH="/volpilcam/user1x/users/echiarui/assignment/test1/masurca_config"
- echo 'pe 350 50' >> meanAndStdevByPrefix.pe.txt
 rename_filter_fastq 'pe' \
 <(exec expand_fastq
 '/voletu/users/echiarui/assignment/sample_data/HS7_R1.fastq.gz' | awk '{if(length(\$0)>250) print substr(\$0,1,250); else print \$0;})' \
 <(exec expand_fastq
 '/voletu/users/echiarui/assignment/sample_data/HS7_R2.fastq.gz' | awk '{if(length(\$0)>250) print substr(\$0,1,250); else print \$0;})' \
 > 'pe.renamed.fastq'
- ./mega_reads_assemble_cluster2_patched.sh -k \$KMER -F -E
 SGE -Pb 5000000000 -q all.q -G 0 -C 25 -t 4 -e
 \$ESTIMATED_GENOME_SIZE -m work1 -a
 /opt/miniconda3/envs/bioinformatics/bin -o "
 cgwErrorRate=0.15" -B 15 -M 17 -D 0.02 -p
 /voletu/users/echiarui/assignment/sample_data/HS7_pacbioD
 ata.fastq.gz

Reason for Parameters: Similar to the rationale for masurca_config, the parameters for assemble_patched.sh were changed for the same reasons. PE was changed to 350 and 50 to account for the insert size of the reads (350bp) and the deviation (± 50). This is also to indicate that HS7_R1 and HS7_R2 are paired end reads. PACBIO was changed to the HS7_pacbioData provided. Rename_filter_fastq was changed to now accept two fastq files for HS7_R1 and HS7_R2.

```
# Code to create run_masurca.sh
touch run_masurca.sh
chmod 755 run_masurca.sh
vim run_masurca.sh
```

Bash script: run_masurca.sh

```
#!/bin/bash
conda run -n bioinformatics bash ./assemble_patched.sh
```

```
# Command to run in terminal
nohup bash run_masurca.sh > masurca.log 2>&1 &
```

4.2 HybridSPAdes

HybridSPAdes is a hybrid genome assembly algorithm which integrates long reads into the assembly process of the SPAdes assembler, a genome assembler built for short reads. The algorithm first builds a rough draft of the genome using short, accurate reads and then uses long reads to fill the gaps and resolve any repeat regions.

It does this by first building a de Bruijn graph from all the short reads. This graph represents all the overlaps between the short reads and it is then simplified into a more manageable assembly graph. The unambiguous regions of the genome will become edges and repeats will become complex branching junctions. The algorithm will then map each long read to the graph with the goal of finding the most likely path of edges in the graph based on how the long read spans the edges.

The use of long reads can also help to resolve “coverage gaps” from the short reads due to disproportionate coverage. It helps by being able to span the gap between short reads and the algorithm will then form a consensus sequence from the long reads to accurately fill the missing sequences and merge two flanking contigs. Long reads are also valuable in being able to resolve repeat regions by enabling the generation of read paths which provide the scaffold to untangle such repetitive regions. This results in the creation of longer and more complete contigs to form the genome (Antipov et al., 2016).

Below illustrates the code that was used to run HybridSPAdes on the short and long reads.

```
# Setting up directories and bash scripts for HybridSPAdes
cd
cd assignments
mkdir test2
cd test2
touch run_spades_hybrid.sh
chmod 755 run_spades_hybrid.sh
vim run_spades_hybrid.sh
```

Bash script: run_spades_hybrid.sh

```
#!/bin/bash
conda run -n bioinformatics spades.py \
    -1
/voletu/users/echiarui/assignment/sample_data/HS7_R1.fastq.gz \
    -2
/voletu/users/echiarui/assignment/sample_data/HS7_R2.fastq.gz \
    --pacbio
/voletu/users/echiarui/assignment/sample_data/HS7_pacbioData.fastq
.gz \
    -o HS7_spades \
    -t 4
```

Reason for Parameters: The parameters were based on the same code provided in Practical 6 but HybridSPAdes was adapted to include a -1 and -2 flag to account for the paired end reads provided. Thread count of 4 is replicated from what was taught and is sufficient for this analysis.

```
# Command to run in terminal
nohup bash run_spades_hybrid.sh > spades.log 2>&1 &
```

4.3 DBG2OLC with SPAdes

DBG2OLC is a hybrid genome assembly algorithm that is built to assemble large genomes from short, accurate reads and long, erroneous reads. It does this by using pre-assembled short-read contigs as a backbone for the long reads. For this assignment, the SPAdes algorithm to generate these pre-assembled short-read contigs from the paired end short reads provided (Prjibelski et al., 2020).

By using short read contigs as a backbone, it helps to simplify and structure long reads. This results in the problem being originally trying to decipher a complex De Bruijn Graph, to a more manageable overlap layout consensus framework. This enables them to speed up the process of genome assembly because they are not performing the computationally expensive step of

base-by-base error correction on the long reads with the short reads. Instead, the process is “correction-free”.

This is done by using the short-read contigs and mapping them onto long, erroneous reads. Each long read is then converted into an ordered list of which contigs it contains which reduces the long, noisy sequences into a short list of anchors. This step is followed up by multiple sequence alignment on the list to identify and remove any structural errors like chimeric reads.

The algorithm will then build the best overlap graph using the cleaned list. The algorithm then traces linear paths through the overlap graph to create a structural backbone for the final assembly. Finally, the algorithm converts the list back into read form which is then laid out according and a consensus sequence is generated to produce a polished final assembly (Ye et al., 2016).

```
# Setting up directories and bash scripts for DBG2OLC with SPAdes
cd
cd assignments
mkdir test3
cd test3
touch run_spades.sh
chmod 755 run_spades.sh
vim run_spades.sh
```

Bash script: run_spades.sh

```
#!/bin/bash
# Defining variables
R1="/voletu/users/echiarui/assignment/sample_data/HS7_R1.fastq.gz"
R2="/voletu/users/echiarui/assignment/sample_data/HS7_R2.fastq.gz"

# Running Spades on Paired End reads
conda run -n bioinformatics spades.py \
-1 ${R1} \
-2 ${R2} \
-o spades_assembly \
-t 4
```

```
# Command to run in terminal
nohup bash run_spades.sh > spades.log 2>&1 &
```

Reason for using SPAdes: SPAdes was used because it was shown to be the best short read assembler for paired end reads during Practical 3. SPAdes is also a De Bruijn graph assembler for short reads, similar to SparseAssembler which was used for DBG2OLC. Another reason that SparseAssembler was not used is that it is not built to handle or utilise the paired end

information provided by the two paired end short read files. Therefore, it felt more fitting for this project to use a short read assembler that would better handle the paired end short read information, convert it into contigs, which can then be used with the DBG2OLC algorithm to generate the final assembly. The parameters used are the same parameters used in the practical and in test 2.

```
# Code to create run_DBG2OLC.sh
touch run_DBG2OLC.sh
chmod 755 run_DBG2OLC.sh
vim run_DBG2OLC.sh
```

Bash script: run_DBG2OLC.sh

```
#!/bin/bash
# Defining Variables
SR="/voletu/users/echiarui/assignment/test3/spades_SR/spades_assembly/contigs.fasta"
LR="/voletu/users/echiarui/assignment/sample_data/HS7_pacbioData.fastq"
conda run -n bioinformatics DBG2OLC \
LD1 0 \
Contigs "$SR" \
k 17 \
AdaptiveTh 0.0001 \
KmerCovTh 2 \
MinOverlap 20 \
RemoveChimera 1 \
f "$LR"
```

```
# Command to run in terminal
nohup bash run_DBG2OLC.sh > DBG2OLC.log 2>&1 &
```

Reason for Parameters: All the parameters used for DBG2OLC were found from Practical 6 but LD1 0 was used in order to additionally indicate to the DBG2OLC algorithm to not filter connections based on LD1 value, the other settings like KmerCovTh and AdaptiveTh would instead be the determining factors for if a connection is retained.

```
# Code to create run_consensus.sh
touch run_consensus.sh
chmod 755 run_consensus.sh
vim run_consensus.sh
```

Bash script: run_consensus.sh

```
#!/bin/bash
# Defining Variables
SR="/voletu/users/echiarui/assignment/test3/spades_SR/spades_assembly/contigs.fasta"
LR="/voletu/users/echiarui/assignment/sample_data/HS7_pacbioData.fasta"
base="/voletu/users/echiarui/assignment/test3"

# Code to call the consensus information from DBG2OLC
cat "$SR" "$LR" > ${base}/ctg_pb.fasta
conda run -n py2 split_and_run_sparc.sh \
"${base}/backbone_raw.fasta \
"${base}/DBG2OLC_Consensus_info.txt \
"${base}/ctg_pb.fasta" \
"${base}/consensusOUT"
```

```
# Command to run in terminal
nohup bash run_consensus.sh > consensus.log 2>&1 &
```

Reason for Parameters: All the parameters used were taken from Practical 6.

4.4 Pilon

Pilon is an assembly polishing method which improve the quality of the genome assembly. It aims to make genomes more contiguous, more complete and have fewer errors. This results in a more accurate genomes (Walker et al., 2014).

```
# Run these commands in the terminal
cd ~/assignment/test1/ # Replace test1 with whichever test is
being analysed

ln -s /path/to/fasta/file final_assembly.fasta # Replace
/path/to/fasta/file with the path to the file for the test being
analysed

conda run -n bioinformatics3 bwa index final_assembly.fasta

conda run -n bioinformatics3 bash -c '
bwa mem -t 4 final_assembly.fasta \
/voletu/users/echiarui/assignment/sample_data/HS7_R1.fastq.gz \
/voletu/users/echiarui/assignment/sample_data/HS7_R2.fastq.gz |
samtools view -b -o HS7_aligns.bam'
```

```
# Code to create run_pilon.sh
touch run_pilon.sh
chmod 755 run_pilon.sh
vim run_pilon.sh
```

Bash script: run_pilon.sh

```
#!/bin/bash
conda run -n bioinformatics3 samtools sort HS7_aligns.bam -o
HS7_alignsSorted.bam

conda run -n bioinformatics3 samtools index HS7_alignsSorted.bam

conda run -n bioinformatics3 pilon \
    --genome final_assembly.fasta \
    --frags HS7_alignsSorted.bam \
    --output HS7_piloned \
    --tracks
```

```
# Command to run in terminal
nohup bash run_pilon.sh > pilon.log 2>&1 &
```

This code was used on all three hybrid assemblies to refine and improve the accuracy of the genome assembly.

5 Assembly Quality Control

For assembly quality control, tools such as gn timer tools, quast and merqu timer was used to determine the quality of the genome assemblies from the various methods.

5.1 gn timer tools and quast

gn timer tools and quast are used to generate statistical metrics like N50 and longest contig to determine which genome assembly tool statistically created the best assembly.

The code used to generate the gn timer tools and Quast results are as shown below:

```
# Run the code below in the terminal for gn timer tools
conda deactivate

# Run gn timer tools on assembly files
java -jar /opt/bioinformatics/gn timer tools/gn timer.jar
/path/to/fasta/file # Replace /path/to/fasta/file with the file
path of the various genome assembly fasta output
```

```
# Entering the right directory
cd
cd ~/assignments/test1 # Replace test1 with whichever test is
being used

# Run the code below in the terminal for quast
conda activate bioinformatics

# Run QUAST on assembly files (Replace /path/to/fasta/file with
the file path to the fasta file generated by the various tools)
quast.py /path/to/fasta/file -o quast_results

# View results
cat quast_results/report.txt
```

a)

Raw
assembly

Post Pilon
assembly
polishing

Results for assembly.scaffolds.fasta
Total number of sequences: 53
Total length of sequences: 4971153 bp
Shortest sequence length : 2552 bp
Longest sequence length : 367974 bp
Total number of Ns in sequences: 0
N50: 159039 (11 sequences) (2616447 bp combined)

Results for test1/pilon/H57_piloned.fasta
Total number of sequences: 53
Total length of sequences: 4970715 bp
Shortest sequence length : 2552 bp
Longest sequence length : 367972 bp
Total number of Ns in sequences: 0
N50: 159034 (11 sequences) (2616194 bp combined)

Results for test2/H57_spades/scaffolds.fasta
Total number of sequences: 657
Total length of sequences: 4941462 bp
Shortest sequence length : 56 bp
Longest sequence length : 334012 bp
Total number of Ns in sequences: 1057
N50: 86699 (19 sequences) (2498660 bp combined)

Results for test2/pilon/H57_piloned.fasta
Total number of sequences: 657
Total length of sequences: 4941691 bp
Shortest sequence length : 56 bp
Longest sequence length : 334012 bp
Total number of Ns in sequences: 952
N50: 86699 (19 sequences) (2498650 bp combined)

Results for test3/consensusOUT/final_assembly.fasta
Total number of sequences: 29
Total length of sequences: 2438638 bp
Shortest sequence length : 902 bp
Longest sequence length : 289009 bp
Total number of Ns in sequences: 0
N50: 107939 (7 sequences) (1230390 bp combined)

Results for test3/pilon/H57_piloned.fasta
Total number of sequences: 29
Total length of sequences: 2428381 bp
Shortest sequence length : 902 bp
Longest sequence length : 287755 bp
Total number of Ns in sequences: 0
N50: 107438 (7 sequences) (1225118 bp combined)

b)

Raw
assembly

Post Pilon
assembly
polishing

MaSuRCA

HybridSPAdes

DBG2OLC + SPAdes

contigs 53
Largest contig 367974
Total length 4971153
GC (%) 36.25
N50 159039
N90 41718
auN 175612.7
L50 11
L90 33
N's per 100 kbp 0.00

contigs 114
Largest contig 334012
Total length 4887624
GC (%) 36.15
N50 86699
N90 21798
auN 99461.0
L50 19
L90 66
N's per 100 kbp 21.63

contigs 29
Largest contig 289009
Total length 2438638
GC (%) 36.36
N50 107939
N90 50895
auN 131930.7
L50 7
L90 20
N's per 100 kbp 0.00

contigs 53
Largest contig 367972
Total length 4970715
GC (%) 36.25
N50 159034
N90 41718
auN 175595.9
L50 11
L90 33
N's per 100 kbp 0.00

contigs 114
Largest contig 334012
Total length 4887853
GC (%) 36.15
N50 86699
N90 21823
auN 99461.0
L50 19
L90 66
N's per 100 kbp 19.48

contigs 29
Largest contig 287755
Total length 2428381
GC (%) 36.42
N50 107438
N90 50663
auN 131369.8
L50 7
L90 20
N's per 100 kbp 0.00

Figure 5: Image depicting the outputs from a) gnx-tools and b) Quast for the three hybrid genome assembly methods, MaSuRCA, HybridSPAdes and DBG2OLC (with SPAdes).

Based on Figure 5, it shows that MaSuRCA has the best N50 value at 159,039, followed by DBG2OLC at 107,939 and then HybridSPAdes at 86,699. These are based on the raw assembly results from both gn x-tools and quast.

MaSuRCA also has the longest contig length at 367, 974, followed by HybridSPAdes at 334,012 and then DBG2OLC at 289,009. These are based on the raw assembly results from both gn x-tools and quast.

DBG2OLC has the least number of contigs at 29, compared to MaSuRCA at 53 and HybridSPAdes at 114. However, DBG2OLC also has the least total length at 2,438,638, followed

by DBG2OLC at 4,887,853 and then MaSuRCA at 4,971,153. This indicates that likely why DBG2OLC has lesser number of contigs compared to the other hybrid assemblers is because it has around half of the total length that the other two hybrid assemblers individually have. The longest contig from DBG2OLC is also less than that of the MaSuRCA and HybridSPAdes. This indicates that the reason why DBG2OLC has lesser contigs is due to it having lesser total length than the other hybrid assemblers.

These results also show that HybridSPAdes has a non-zero value for the # N's per 100kbp. This means that the assembly has an average of 21.63 uncalled or ambiguous bases per 100,000 bases in the genome assembly. This illustrates that the quality of the resultant genome assembly generated by HybridSPAdes is not very accurate compared to MaSuRCA and DBG2OLC which has zero for the # N's per 100kbp row in comparison.

5.2 Merqury

The code below was used to generate the output for merqury

```
# Setting up directory
cd
mkdir ~/assignments/test1/merqury # Replace test1 with whichever
test is being used
cd ~/assignments/test1/merqury

conda activate bioinformatics2

# Generate kmer counts from reads
meryl k=23 count output pe_reads23.meryl
~/assignments/sample_data/HS7_R?.fastq.gz

# Compare the kmer obtained from the reads to the assembled genome
merqury.sh pe_reads23.meryl /path/to/fasta/file pe_mp23

# Generate a kmer plot
Rscript
/opt/miniconda3/envs/bioinformatics2/share/merqury/plot/plot_spect
ra_cn.R -f pe_mp23.scaffolds.spectra-cn.hist -o pe_mp23

# Visualise the plot
xdg-open pe_mp23.scaffolds.spectra-cn.st.png

# View completeness statistics
cat /path/to/file_completeness.stats
```

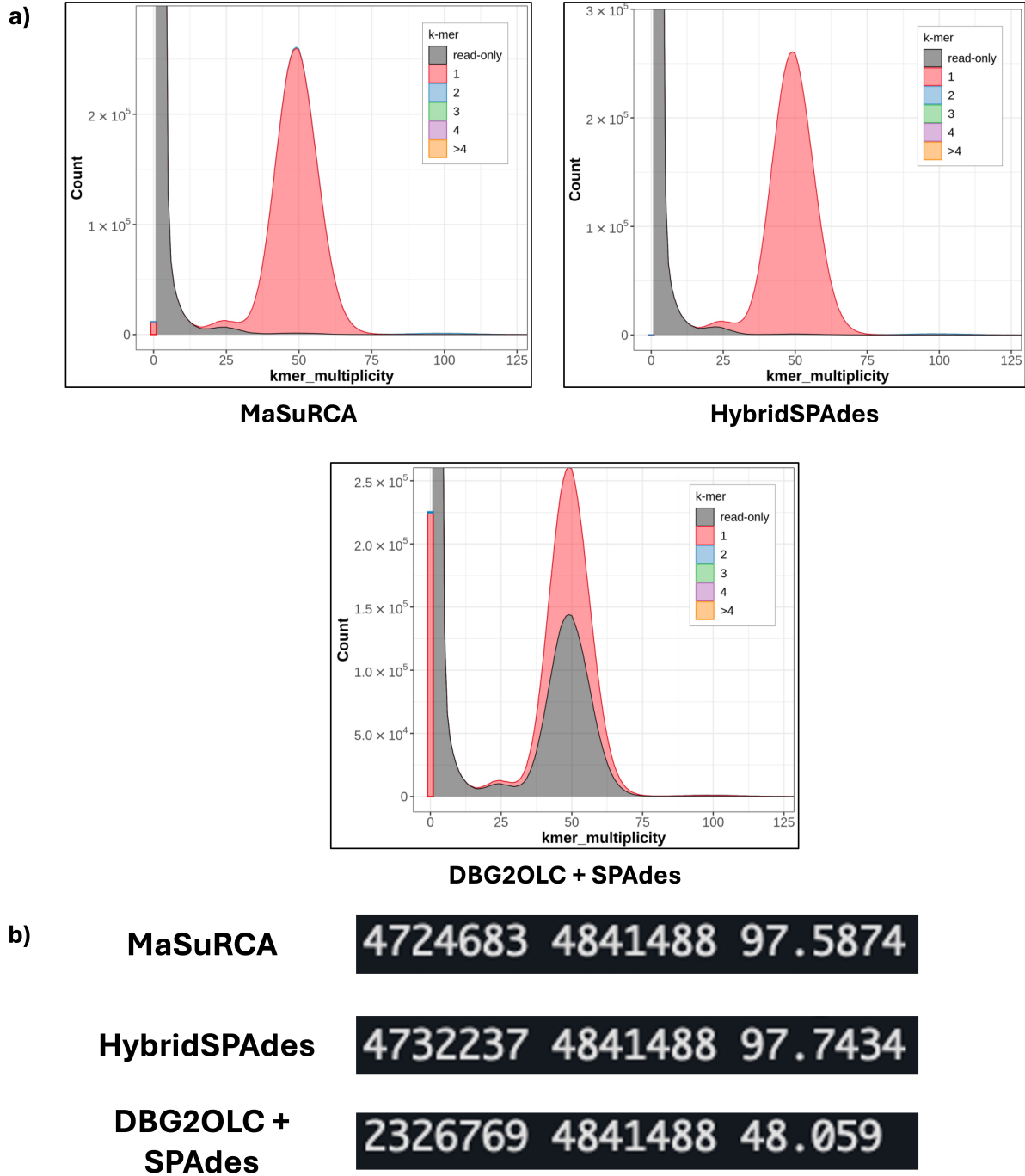


Figure 6: Image depicting the outputs from merquary such as the a) Visualised plots and b) Completeness statistics for MaSuRCA, HybridSPAdes and DBG2OLC (with SPAdes) respectively.

Based on Figure 6, we can observe the graphs for the three hybrid assemblers and conclude that all three did a good job of removing error-kmers at the left side of the plot and almost all kmers appear only once in the assembly.

Looking closer at the MaSuRCA merquary graph, it can be observed that there is some duplication involved in the assembly as there is a small blue portion at the peak of the graph.

This indicates the presence of some repetitive kmers. This could explain why it has a lower completeness score compared to the HybridSPAdes method, which does not appear to have such duplication.

However, it is notable that DBG2OLC has a completeness statistic of 48.059% which is the lowest among the three hybrid assemblers and this is because it classified a group of kmers at kmer multiplicity of around 50 as error kmers when they should not be classified as such.

In terms of completeness statistics, HybridSPAdes showed the highest percentage at 97.734%, followed by MaSuRCA at 97.5874% and DBG2OLC at 48.059%.

5.3 Conclusion

In conclusion, the best assembly among the three hybrid assemblies that were tested is MaSuRCA.

The MaSuRCA algorithm was chosen because its resulting genome assembly was shown in Figure 5 to have the best N50 value and longest contig length. It was also shown to have the least number of contigs when not accounting for DBG2OLC which has a way shorter sequence. It also has zero uncalled or ambiguous bases per 100,000 bases in the genome assembly.

For the quality evaluation of MaSuRCA, as shown in Figure 6, it showed that the error-kmers were removed from the assembly and the completeness statistics were above 90% and though it was not the highest completeness percentage among the three hybrid assemblies, it was very close to the best completeness percentage assembly method which was HybridSPAdes with only a difference of 0.156%.

Therefore, with all of these into consideration, MaSuRCA should be considered the best hybrid assembler among the three methods for this genome being analysed.

References

- Antipov, D., Korobeynikov, A., McLean, J. S., & Pevzner, P. A. (2016). hybridSPAdes: an algorithm for hybrid assembly of short and long reads. *Bioinformatics*, 32(7), 1009–1015. <https://doi.org/10.1093/BIOINFORMATICS/BTV688>
- Marçais, G., & Kingsford, C. (2011). A fast, lock-free approach for efficient parallel counting of occurrences of k-mers. *Bioinformatics (Oxford, England)*, 27(6), 764–770. <https://doi.org/10.1093/BIOINFORMATICS/BTR011>
- Prjibelski, A., Antipov, D., Meleshko, D., Lapidus, A., & Korobeynikov, A. (2020). Using SPAdes De Novo Assembler. *Current Protocols in Bioinformatics*, 70(1), e102. <https://doi.org/10.1002/CPBI.102;PAGE:STRING:ARTICLE/CHAPTER>
- Ye, C., Hill, C. M., Wu, S., Ruan, J., & Ma, Z. (2016). DBG2OLC: Efficient Assembly of Large Genomes Using Long Erroneous Reads of the Third Generation Sequencing Technologies. *Scientific Reports 2016 6:1*, 6(1), 31900-. <https://doi.org/10.1038/srep31900>
- Zimin, A. V., Puiu, D., Luo, M. C., Zhu, T., Koren, S., Marçais, G., Yorke, J. A., Dvořák, J., & Salzberg, S. L. (2017). Hybrid assembly of the large and highly repetitive genome of *Aegilops tauschii*, a progenitor of bread wheat, with the MaSuRCA mega-reads algorithm. *Genome Research*, 27(5), 787–792. <https://doi.org/10.1101/GR.213405.116>